Tema 3 — Web and Service Application Architectures

Resum de `grau_asw03_architecture.pdf` (37 diapositives), **amb les figures incloses**, i una bateria de preguntes probables al final.

> **Context:** el professor ha dit que al primer control hi caurà **una pregunta d'1,5 punts del tema 3**. Als 8 controls anteriors que hi ha a `control1/` **no hi ha cap pregunta de teoria** — tots són només l'exercici de disseny de 7 punts — o sigui que aquesta és nova i no es pot deduir d'exàmens passats. Les preguntes de la secció final són **predicció meva** a partir de quines parts de les transparències tenen definició tancada, llista numerada o contrast explícit.

---

1. Introducció

Software architecture (slide 3) — definició literal

> *Software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.*

Tres ingredients: **elements**, **relacions** i **propietats** d'ambdós.

Architectural viewpoints (slide 4) — la distinció clau

	Què és	Decideix el desplegament?
Logical view	Organització a gran escala del programari en **packages, subsystems i layers** que separen lògicament la funcionalitat	**No.** Per això es diu lògica
Physical view	Organització i **distribució de l'arquitectura lògica entre nodes computacionals** d'una xarxa	Sí

Vocabulari: la vista lògica es compta en **layers** (capes), la física en **tiers**. La slide 5 en dona l'exemple: *3-Layered Logical Architecture* al costat de *2(+)-Tiered Client/Server Physical Architecture*. **No hi ha correspondència 1 a 1**: 3 capes lògiques poden desplegar-se en 2 tiers físics.

---

2. Arquitectures d'aplicacions web

Les dues figures de partida (slides 7 i 8)

Les dues tenen **la mateixa business layer, data layer i cross-cutting**. El que canvia és **només el pis de dalt**.

	"Classic" Web Apps (slide 7)	Single-Page Apps (slide 8)
Client	`WEB BROWSER` amb una sola caixa: **Rendering / DOM**	`WEB BROWSER` amb un **JavaScript Runtime** (*Presentation: components, routing* + *Business Processing Components*) i **Web Storage / IndexedDB**
Capa de dalt del servidor	**PRESENTATION LAYER**: UI Components (views, templates) + UI Process Components (controllers)	**SERVICES LAYER**: Service Interfaces (REST/GraphQL) + Message Types (DTOs)
Business layer	Application Facade · Business Workflows · Business Components · Business Entities	**Idèntica**
Data layer	Data Access Components · Data Helpers/Utilities · Service Agents	**Idèntica**
Cross-cutting	Security · Operational Management · Communication	**Idèntica**

La presentació no desapareix: es trasllada al client. I el servidor deixa d'enviar HTML i passa a enviar **dades (DTOs)** per una API.

Dimensions en el disseny de l'arquitectura física (slide 9)

Tres grups: **requisits no funcionals**, **restriccions**, i **escenaris alternatius de desplegament**.

Requisits no funcionals (slide 10) — 5

Requisit	Contingut
Performance	Aguantar la càrrega: màxim d'usuaris concurrents, peticions per unitat de temps, temps màxim per lliurar una pàgina
Scalability	Fàcilment extensible, **idealment afegint més instàncies** (escalat **horitzontal**)
Availability	Les fallades no han d'afectar significativament el servei
State maintenance (reliability)	L'estat de la interacció s'ha de preservar **fins i tot** amb l'aplicació distribuïda o amb fallades
Security	Dades protegides; usuaris identificats i amb accés **només** al que tenen dret

Restriccions (slide 11) — 3

- **Cost**: processadors, xarxa, interfícies, llicències o subscripcions al núvol.
- **Complexity**: algunes configuracions són més simples de muntar i mantenir; la **manca o el cost de perfils especialitzats** també hi compta.
- **Corporate standards and infrastructures**.

Escenaris de desplegament (slide 12) — 3

Escenari	Equipament	Qui l'opera
On-premises	De l'organització, al seu centre de dades	El seu IT intern
Co-located	De l'organització, **en un centre de dades d'un tercer**	Encara el seu IT intern
Cloud	Del proveïdor	El proveïdor, **on demand** (IaaS o PaaS); l'aplicació sol anar en **contenidors**

El *co-located* és el que es falla: l'equipament és teu i el gestiones tu; el que llogues és **només l'espai**.

---

3. Els patrons d'arquitectura física

Les figures són diagrames de desplegament UML, amb estereotips `«device»`, `«execution environment»` i `«artifact»`. Les comunicacions van etiquetades (`«HTTPS»`, `«SQL / TLS»`), i els tallafocs són **línies verticals discontínues vermelles** que delimiten la zona marcada com a **«Perimeter network (DMZ)»**.

Patró 1 — Single Server (slides 13-15)



Els tres entorns d'execució dins d'una sola màquina, lligats amb `«use»`.

Dimensió	Avaluació
Performance	Depèn de la configuració: CPU, memòria, latència de disc. **El DBMS és intensiu en memòria i CPU**
Scalability	**Limitada per l'arquitectura de maquinari del servidor**
Availability	**Tot element és un single point of failure.** Es pot millorar amb maquinari redundant (múltiples CPU, discos en mirall) i múltiples processos
State maintenance	**Cap problema** amb cap de les tres opcions (client, servidor, base de dades)
Security	**Punt feble.** Qui trenca el tallafocs i el servidor web té **accés directe a la base de dades**
Cost / Complexity	**Baixos** tots dos

Patró 2 — Separate Database (slides 16-17)

La base de dades passa a un `«device» :Database Server` propi. L'`:Application Server` queda a la **DMZ**, entre dos tallafocs; la base de dades, darrere l'intern. Comunicació `«SQL / TLS»`.

Dimensió	Avaluació
Performance	**Millor**: una màquina extra, i cada tier s'afina al seu programari
Scalability	**Més escalable**: s'actua separadament sobre cada tier. **El primer coll d'ampolla és al middle tier**
Availability	**NO es millora.** Cada component segueix sent un SPOF
Security	**Millorada significativament**: el tallafocs intern pot **prohibir del tot les peticions HTTPS** i deixar passar només peticions de base de dades

> **La trampa més probable de tot el tema: separar la base de dades NO millora la disponibilitat.**

Patró 3 — Replicated Web Server (slides 18-19)

Apareix un `«device» :Load Balancer` i els `:Application Server` es dibuixen **apilats amb la multiplicitat** `[1..n]`. El balancejador porta l'anotació **«TLS termination and re-encryption»**.

Dimensió	Avaluació
Performance i scalability	**Millorades**: **load balancing** i **clustering** (grup de nodes amb vista unificada)
Availability	**Millorada**: **fail-over** — si cau un node, la càrrega es redistribueix
State maintenance	**Session affinity** (*sticky sessions*): el balancejador envia tota la sessió **al mateix servidor**. O **session migration**: l'estat **es comparteix** entre nodes
Security	Millora la **transmissió**: el TLS **es termina al balancejador** i es torna a xifrar cap als servidors

Patró 4 — Separate Static and Dynamic Content (slides 20-21)

El balancejador **encamina per path**:



Fixa't en les multiplicitats: `[1..n]` i `[1..m]`, lletres diferents. És el dibuix que demostra que els dos tiers **s'escalen independentment**.

- Millora **performance, scalability i availability**. Una configuració ben equilibrada sol demanar **més instàncies per al tier dinàmic**.
- Els recursos estàtics es **descarreguen a una CDN**, cosa que treu aquell trànsit de l'origen.
- **El cost addicional és operatiu**: dues unitats de desplegament a construir, publicar i monitorar.

Patró 5 — Web Caching (slides 22-25)

Definició: emmagatzemar temporalment recursos en una ubicació d'accés ràpid per recuperar-los més tard.

Beneficis: reduir el temps de resposta i l'**esforç de càlcul** quan el recurs es construeix dinàmicament.

Què es pot cachejar (4): pàgines HTML estàtiques i multimèdia · **fragments** de pàgina calculats per l'aplicació · **dades intermèdies** (p. ex. JSON) · el **resultat de consultes** a la base de dades.

On es pot cachejar (4):

Lloc	Descripció
Browser cache	Tot navegador en té una, de pàgines i fitxers multimèdia
Proxy cache	Còpia local dels recursos demanats, evita l'accés a Internet. **Avui són rares, perquè el trànsit va xifrat amb HTTPS**
Reverse proxy cache	**Davant d'un clúster**: intercepta peticions, cacheja els objectes que produeixen els servidors. Inclou **page prefetching**: a partir de l'última petició, carrega a la cache les pàgines més probables
CDN	Externalitza la cache a una xarxa de servidors per tot Internet

Productes: davant dels servidors → **Varnish, NGINX, Apache Traffic Server**. Al costat de l'aplicació, per a resultats de consultes i dades de sessió → **Redis, Memcached**.

CDN (slide 25): les peticions es ruten al **PoP** (*point of presence*) més proper **per resolució DNS o anycast routing**; la CDN pot executar **codi a l'edge** (*edge functions*). Proveïdors: Akamai, Cloudflare, Amazon CloudFront, Fastly.

---

4. Arquitectures d'aplicacions de servei (slides 26-31)

La figura (slide 27)

Igual que la de les SPA, **però a dalt no hi ha cap navegador**: hi ha `EXTERNAL SYSTEMS` **amb** `Service Consumers`. L'aplicació de servei **no té capa de client**: el consumidor és un altre sistema.

A sota, el mateix `Application Server` amb **SERVICES LAYER** (Service Interfaces REST/GraphQL + Message Types DTOs), **BUSINESS LAYER** i **DATA LAYER**, i el mateix cross-cutting.

Les tres afirmacions (slides 28-29)

1. Els **requisits no funcionals, les restriccions i els escenaris de desplegament són els mateixos** que per a les aplicacions web.
2. Es poden fer servir **els mateixos patrons**, tenint en compte els problemes de rendiment i seguretat **inherents als escenaris distribuïts**.

3. **El tier de contingut estàtic no és necessari** quan l'aplicació només exposa una API: **totes les peticions van al tier d'aplicació**.
4. Quan **diversos tipus de client** consumeixen els serveis, el patró **Backend for Frontend** afegeix **un tier per experiència de client**.

Patró — Backend for Frontend (slides 30-31)

La figura mostra **tres clients, cadascun amb el seu BFF**, i tots tres sobre **una sola plataforma de serveis compartida**:



Què fa l'Application Runtime de cada BFF (la caixa discontínua de la figura), *«no business logic: it stays in the shared services»*:

- **call aggregation** (agregar crides)
- **response shaping** (donar forma a la resposta)
- **protocol translation**
- **auth & token handling**
- **client-specific caching**

Avaluació (slide 31):

Aspecte	Contingut
Tailored APIs	Cada client rep una resposta a la seva mida: **ni over-fetching ni under-fetching**, i cap API única amb requisits contradictoris
Independent evolution	Cada BFF és propietat, es desplega i s'escala per l'**equip del seu frontend**
Fault isolation	La caiguda d'un BFF **no afecta els altres clients**
Costs i riscos	Més serveis a construir, desplegar, monitorar i assegurar, i lògica que es pot **duplicar**. **Un BFF ha de ser prim**

---

5. Cloud computing (slides 32-36)

Tot ve de la **definició del NIST** (SP 800-145). La xifra: **5 – 3 – 4**.

Definició (slide 33)

> *Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources … that can be rapidly provisioned and released with minimal management effort or service provider interaction.*

5 característiques essencials (slide 34)

1. **On-demand self-service** — aprovisionament unilateral i automàtic, **sense interacció humana**
2. **Broad network access** — per xarxa, amb mecanismes estàndard
3. **Resource pooling** — recursos assignats i reassignats dinàmicament
4. **Rapid elasticity** — aprovisionament ràpid i elàstic
5. **Measured service** — ús monitorat, controlat i informat, amb transparència

Els models de servei (slide 35) — la figura

La diapositiva dibuixa **cinc capes**, però **només tres porten l'etiqueta oficial del NIST** (les de la columna de l'esquerra):

```
        Cloud Clients      ← web browser, mobile app, CLI, API client
              ▲▼
SaaS     ★    SaaS         ← CRM, e-mail, virtual desktop, collaboration
        FaaS / Serverless  ← functions, event handlers, scaling to zero
PaaS     ★    PaaS         ← execution runtime, database, web server, dev tools
        CaaS / Containers  ← container runtime i orquestració (p. ex. Kubernetes)
IaaS     ★    IaaS         ← virtual machines, storage, load balancers, network

 ★ = els 3 models del NIST
```

Peu de la figura: *«upper levels: less to manage — lower levels: more control».*

Dues coses que et poden preguntar i que només es veuen a la figura:

- **FaaS va entre PaaS i SaaS**; **CaaS va entre IaaS i PaaS**.
- L'eix del compromís: **com més amunt, menys has de gestionar; com més avall, més control tens.**

4 models de desplegament (slide 36)

Model	Per a qui
Private cloud	Ús exclusiu d'**una sola organització**
Community cloud	Ús exclusiu d'una **comunitat concreta** d'organitzacions amb **preocupacions compartides**
Public cloud	Ús obert al **públic general**
Hybrid cloud	**Composició de dues o més** infraestructures distintes, unides per tecnologia que permet la **portabilitat de dades i aplicacions**

---

6. Preguntes probables i resposta

Predicció meva. Per a 1,5 punts espera't **una o dues preguntes curtes**, o **un escenari on has de triar patró i justificar-lo**.

Tipus A — Contrast entre patrons (el més probable)

P: Compara el patró Separate Database amb el Single Server. Quins requisits no funcionals millora i quins no?

> **Millora:** el **rendiment** (una màquina extra, i cada tier es pot afinar al programari que hi corre), l'**escalabilitat** (es pot actuar separadament sobre cada tier; el primer coll d'ampolla sol ser al middle tier) i molt especialment la **seguretat** (el tallafocs intern pot prohibir del tot les peticions HTTPS i deixar passar només peticions de base de dades).
>
> **No millora la disponibilitat:** cada component continua sent un *single point of failure*. Per millorar-la cal **replicació**, que és el patró següent, amb *fail-over*.

P: Per què el patró Replicated Web Server sí que millora la disponibilitat?

> Perquè introdueix **clustering** i **fail-over**: si un node del clúster cau, la seva càrrega es redistribueix entre els altres nodes del mateix clúster. Ja no hi ha un únic servidor d'aplicació del qual depengui tot.

Tipus B — Estat de sessió amb servidors replicats

P: Amb servidors replicats, com es manté l'estat de sessió? Explica les dues opcions.

> - **Session affinity** (*sticky sessions*): el **balancejador de càrrega** envia totes les peticions d'una mateixa sessió **al mateix servidor**. Simple, però si aquell node cau es perd la sessió, i desequilibra la càrrega.
> - **Session migration**: l'estat de sessió **es comparteix** entre tots els servidors del clúster. Qualsevol node pot atendre qualsevol petició, però cal un magatzem compartit i té un cost de sincronització.

Tipus C — Caching

P: On es pot cachejar en una arquitectura web? Per què els proxy caches han deixat de fer-se servir?

> Hi ha quatre llocs: la **cache del navegador**, el **proxy cache**, el **reverse proxy cache** (davant d'un clúster de servidors) i la **CDN**.
>
> Els **proxy caches** són rars avui **perquè el trànsit va xifrat amb HTTPS**: un intermediari no pot llegir ni emmagatzemar el contingut d'una connexió que no pot desxifrar.

P: Què és el page prefetching i on viu?

> Viu al **reverse proxy cache**. Consisteix a **carregar a la cache, a partir de l'última petició rebuda, les pàgines que és més probable que es demanin a continuació**.

Tipus D — Lògic vs. físic

P: Explica la diferència entre arquitectura lògica i arquitectura física.

> La **lògica** és l'organització a gran escala del programari en *packages*, *subsystems* i **layers** que separen lògicament la funcionalitat; **no pren cap decisió sobre com es reparteixen els elements entre nodes**.
>
> La **física** és l'organització i **distribució d'aquella arquitectura lògica entre diferents nodes computacionals** d'una xarxa, comptats en **tiers**.
>
> No hi ha correspondència 1 a 1: una arquitectura de **3 capes lògiques** es pot desplegar en **2 tiers físics**.

Tipus E — Cloud

P: Enumera els models de servei i els models de desplegament del cloud computing segons el NIST.

> **Tres models de servei:** IaaS (màquines virtuals, emmagatzematge, balancejadors, xarxa), PaaS (runtime d'execució, base de dades, servidor web, eines de desenvolupament) i SaaS (CRM, correu, escriptori virtual, col·laboració). Entremig, les transparències hi situen també **CaaS / Containers** (entre IaaS i PaaS) i **FaaS / Serverless** (entre PaaS i SaaS).
>
> **Quatre models de desplegament:** private, community, public i hybrid.
>
> El compromís: **com més amunt, menys has de gestionar; com més avall, més control tens.**

P: Diferència entre on-premises, co-located i cloud.

> - **On-premises:** l'aplicació corre al centre de dades **de la pròpia organització** i l'opera el seu **departament IT intern**.
> - **Co-located:** l'equipament **és de l'organització i el gestiona ella**, però està **instal·lat en un centre de dades d'un tercer**. El que es lloga és l'espai.
> - **Cloud:** la infraestructura **i la seva operació** les proporciona un **proveïdor extern**, aprovisionades **on demand** (IaaS o PaaS). L'aplicació normalment s'empaqueta com a **contenidors**.

Tipus F — Serveis i BFF

P: Quines diferències hi ha entre l'arquitectura d'una aplicació web i la d'una aplicació de servei?

> Els **requisits no funcionals, les restriccions i els escenaris de desplegament són els mateixos**, i es poden fer servir **els mateixos patrons**. Les diferències:
>
> - L'aplicació de servei **no té capa de presentació ni capa de client**: a dalt hi ha **sistemes externs** (*service consumers*), i la capa de dalt del servidor és la **services layer** (interfícies REST/GraphQL i DTOs).
> - **El tier de contingut estàtic no és necessari** si només s'exposa una API: totes les peticions van al tier d'aplicació.
> - Cal tenir en compte els problemes de rendiment i seguretat **inherents als escenaris distribuïts**.

P: Què és el patró Backend for Frontend i quan s'aplica?

> S'aplica quan **diversos tipus de client** consumeixen els mateixos serveis, i consisteix a afegir **un tier per experiència de client** (un BFF per al web, un per al mòbil, una API per als socis), tots sobre **una plataforma de serveis compartida**.
>
> Cada BFF fa **agregació de crides, modelat de la resposta, traducció de protocol, gestió d'autenticació i tokens, i cache específica del client** — però **cap lògica de negoci: aquesta es queda als serveis compartits**.
>
> **Avantatges:** APIs a mida (ni *over-fetching* ni *under-fetching*), evolució independent per equip, i aïllament de fallades. **Costos:** més serveis a construir i mantenir, i risc de duplicar lògica.

Tipus G — Escenari (el format que més em preocuparia)

P: Una botiga en línia pateix caigudes quan el seu únic servidor es reinicia, i a les rebaixes no dona l'abast. Quin patró aplicaries i per què?

> **Replicated Web Server.** Resol les dues coses alhora: el **load balancing** i el **clustering** donen rendiment i escalabilitat horitzontal per absorbir el pic de les rebaixes, i el **fail-over** dona disponibilitat, perquè si un node cau la seva càrrega es redistribueix.
>
> A canvi, cal resoldre l'**estat de sessió** entre els nodes (*session affinity* o *session migration*), i el cost i la complexitat pugen.
>
> Si a més la botiga serveix moltes imatges de producte, hi afegiria **Separate Static and Dynamic Content**, amb els recursos estàtics descarregats a una **CDN**, perquè els dos tiers s'escalen independentment i normalment cal més capacitat al dinàmic que a l'estàtic.

La justificació que puntua sempre té la mateixa forma: **quin requisit no funcional millores, i a canvi de què** (cost, complexitat, unitats de desplegament, estat de sessió).

---

7. Resum de les llistes per memoritzar

Llista	Elements
Requisits no funcionals (5)	Performance · Scalability · Availability · State maintenance · Security
Restriccions (3)	Cost · Complexity · Corporate standards
Escenaris de desplegament (3)	On-premises · Co-located · Cloud
Patrons web (5)	Single Server · Separate Database · Replicated Web Server · Separate Static/Dynamic · Web Caching
On cachejar (4)	Browser · Proxy · Reverse proxy · CDN
Què cachejar (4)	HTML i multimèdia · fragments de pàgina · dades intermèdies (JSON) · resultats de consultes
NIST (5-3-4)	5 característiques · 3 models de servei · 4 models de desplegament
Característiques NIST (5)	On-demand self-service · Broad network access · Resource pooling · Rapid elasticity · Measured service
Models de servei (3 + 2)	IaaS · PaaS · SaaS (+ CaaS entre els dos primers, FaaS entre els dos últims)
Models de desplegament (4)	Private · Community · Public · Hybrid
Què fa un BFF (5)	Agregació de crides · modelat de resposta · traducció de protocol · auth i tokens · cache per client